

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250321>

Outline : Convolutional Neural Networks (code [samples](#))

- Definitions / Motivation
- The Architecture of the Visual Cortex
- Convolutional Layer
- Pooling Layer
- CNN Architectures
- Object Detection
- Semantic Segmentation
- Takeaways

—

Source: Aurelien Geron, "Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems", 2019 ([Available online](#)) / **Chapter 14**

Definition / Motivation

Perception largely takes place outside the realm of our consciousness: **specialized** visual, auditory, and other sensory modules in our brains: our consciousness gets “high-level features”.

Convolutional neural networks (CNNs) emerged from the study of the brain’s visual cortex.

CNNs have managed to achieve superhuman performance on some complex visual tasks: image search services, self-driving cars, automatic video classification systems, and more. Also good at as *voice recognition* or *natural language processing* (NLP).

The Architecture of the Visual Cortex (1)

- David H. Hubel and Torsten Wiesel performed a series of experiments on cats in 1958 and 1959 (and a few years later on monkeys), giving crucial insights on the structure of the visual cortex (the authors received the Nobel Prize in Physiology or Medicine in 1981 for their work).
- Many neurons in the visual cortex have a small local receptive field, meaning they react only to visual stimuli located in a limited region of the visual field.
- The receptive fields of different neurons may overlap, and together they tile the whole visual field.
- Some neurons react only to images of horizontal lines, while others react only to lines with different orientations (two neurons may have the same receptive field but react to different line orientations).

The Architecture of the Visual Cortex (2)

- The higher-level neurons are based on the outputs of neighboring lower-level neurons.
- This is a powerful architecture able to detect all sorts of complex patterns in any area of the visual field.

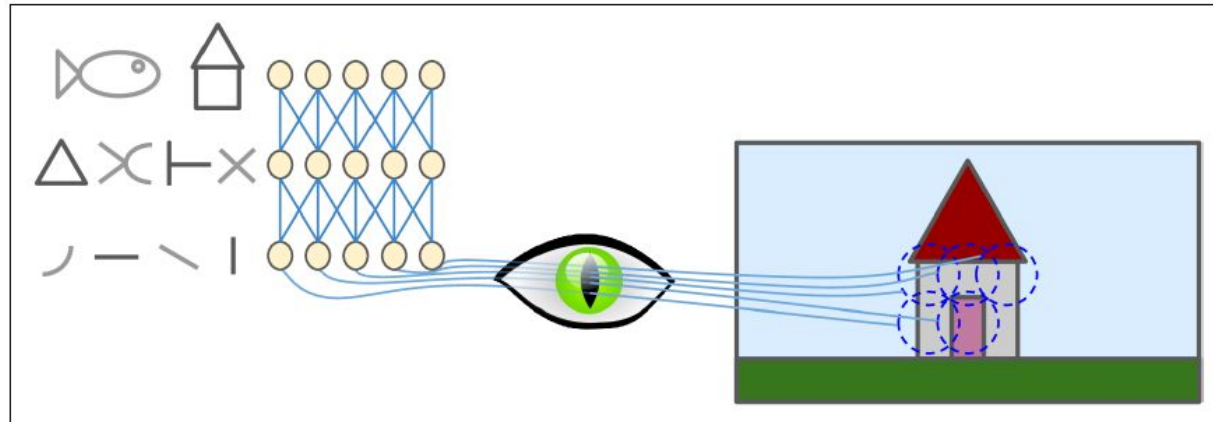


Figure 14-1. Local receptive fields in the visual cortex

The Architecture of the Visual Cortex (3)

- The studies of the visual cortex inspired the neocognitron, introduced in 1980, which gradually evolved into what we now call convolutional neural networks
- An important milestone was a 1998 paper, which introduced the famous *LeNet-5 architecture*, widely used to recognize handwritten check numbers.
 - has fully connected layers.
 - introduces new building blocks: *convolutional layers* and *pooling layers*.

Convolutional Layer (1)

The convolutional layer: neurons in the first convolutional layer are not connected to every single pixel in the input image (like they were in previous chapters), but only to pixels in their receptive fields.

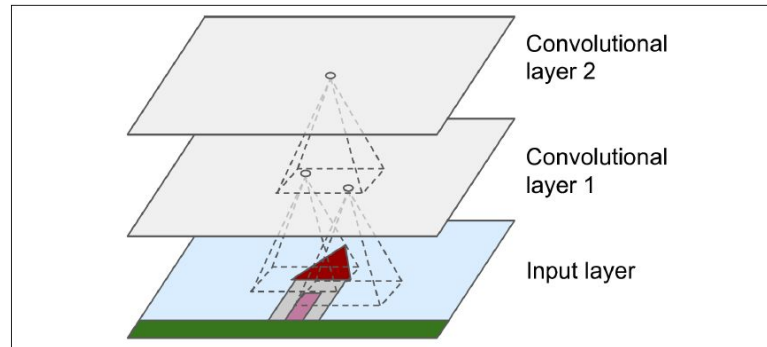


Figure 14-2. CNN layers with rectangular local receptive fields

Convolutional Layer (2)

In turn, each neuron in the second convolutional layer is **connected only to neurons located within a small rectangle in the first layer**. This architecture allows the network to **concentrate on small low-level features** in the first hidden layer, then assemble them into larger higher-level features in the next hidden layer, and so on.

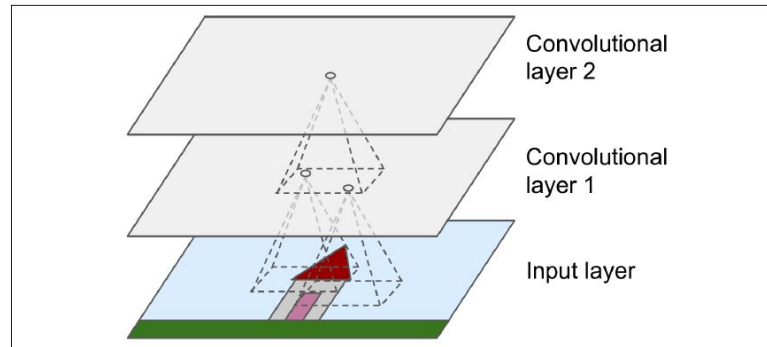


Figure 14-2. CNN layers with rectangular local receptive fields

Convolutional Layer (3)

A neuron located in row i , column j of a given layer is connected to the outputs of the neurons in the previous layer located in rows i to $i + f_h - 1$, columns j to $j + f_w - 1$, where f_h and f_w are the height and width of the receptive field.

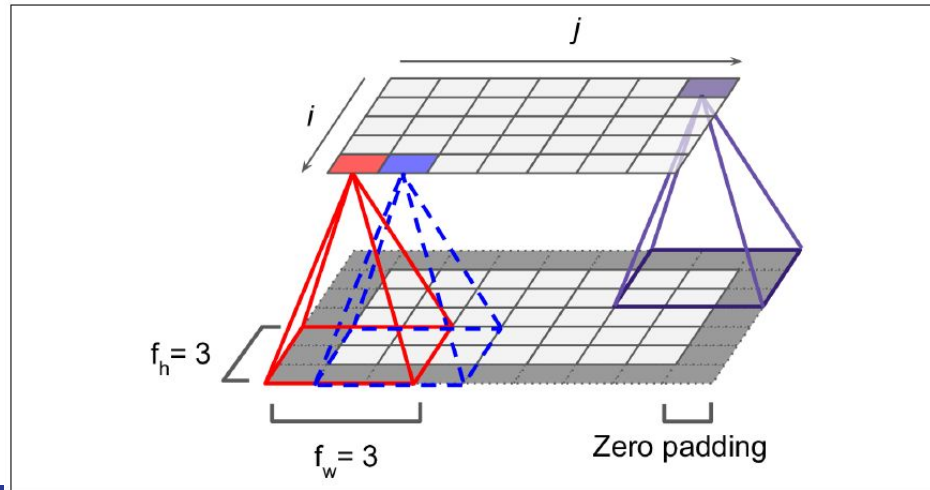


Figure 14-3. Connections between layers and zero padding

Convolutional Layer (4)

In order for a layer to have the same height and width as the previous layer, it is common to add zeros around the inputs, as shown in the diagram. This is called *zero padding*.

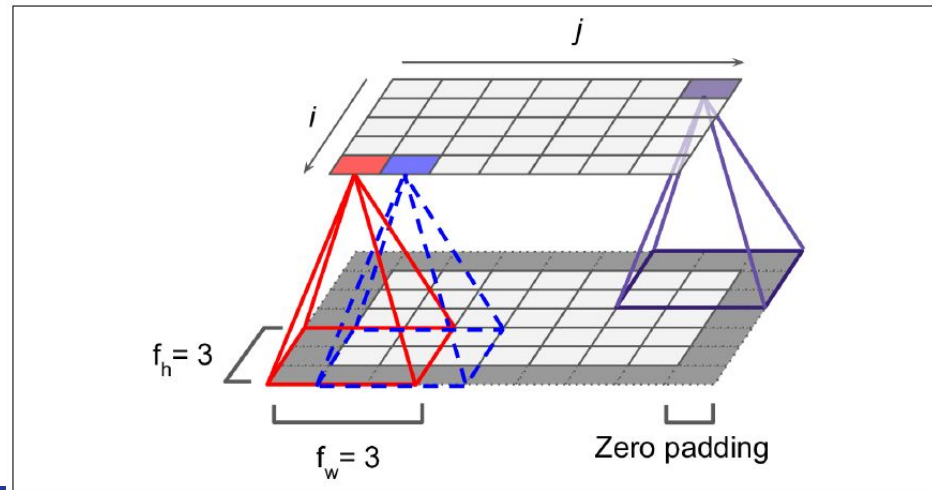


Figure 14-3. Connections between layers and zero padding

Convolutional Layer (5)

Also possible to connect a large input layer to a much smaller layer by spacing out the receptive fields. The shift from one receptive field to the next is called the *stride*.

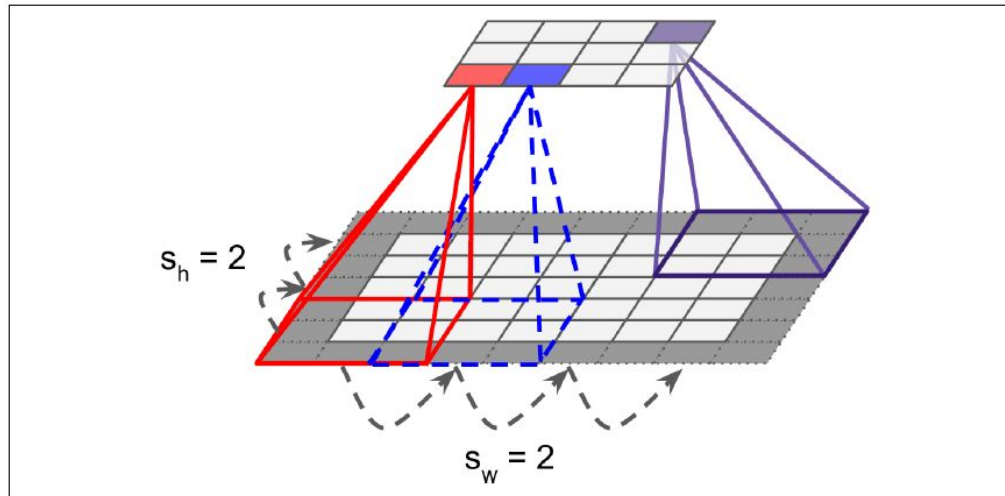


Figure 14-4. Reducing dimensionality using a stride of 2

Convolutional Layer (6)

For example: 5×7 input layer (plus zero padding) connected to a 3×4 layer, using 3×3 receptive fields and a stride of 2 (in this example the stride is the same in both directions, but it does not have to be so) strides.

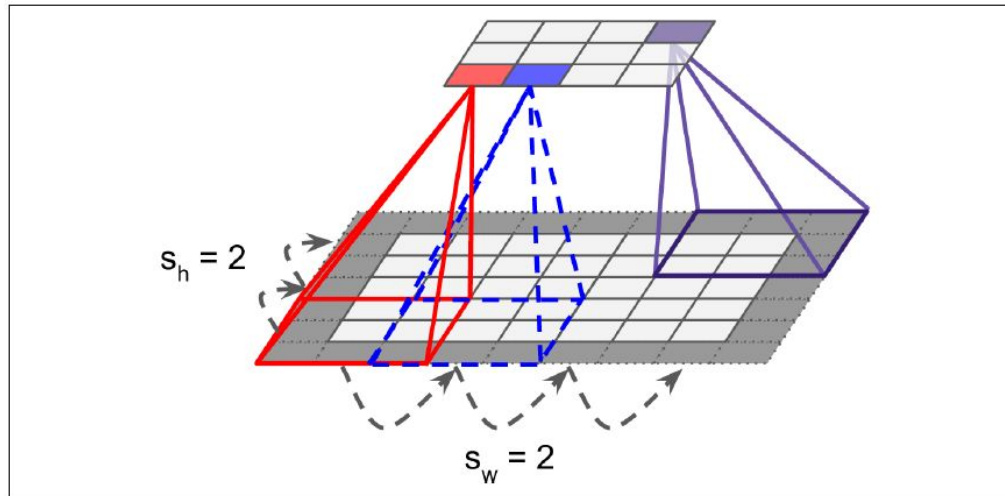


Figure 14-4. Reducing dimensionality using a stride of 2

Convolutional Layer (7)

A neuron located in row i , column j in the upper layer is connected to the outputs of the neurons in the previous layer located in rows $i \times s_h$ to $i \times s_h + f_h - 1$, columns $j \times s_w$ to $j \times s_w + f_w - 1$, where s_h and s_w are the vertical and horizontal strides.

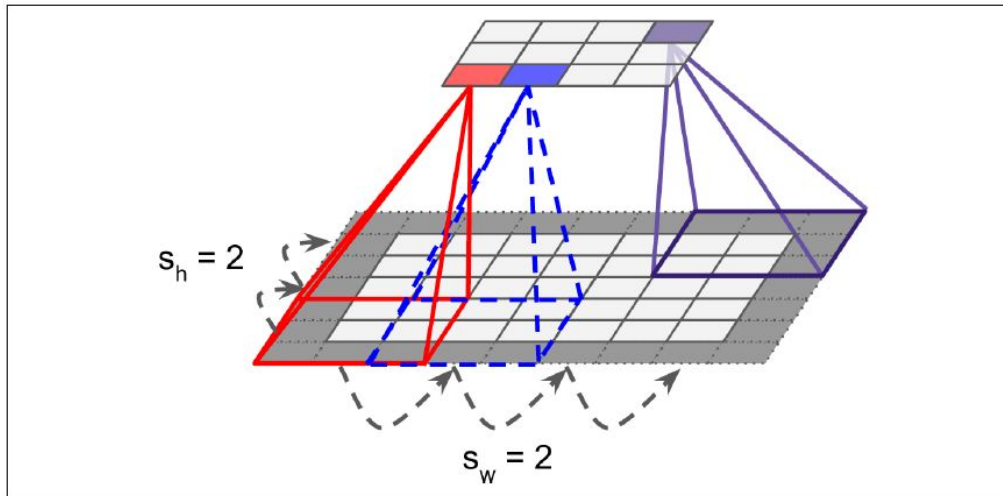


Figure 14-4. Reducing dimensionality using a stride of 2

Filters (1)

A neuron's weights can be represented as a small image the size of the receptive field. For example, below are two possible sets of weights, called *filters* (or *convolution kernels*).

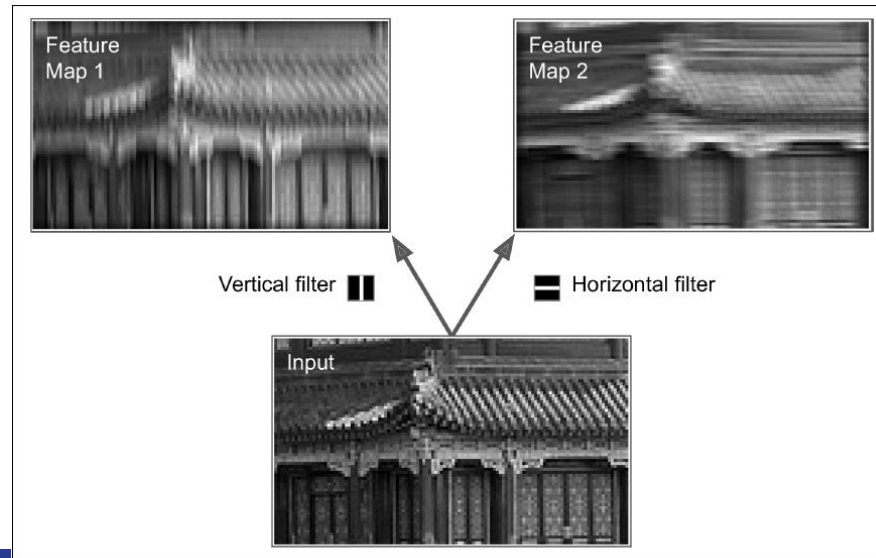


Figure 14-5. Applying two different filters to get two feature maps

Filters (2)

First one: black square with a vertical white line in the middle (7×7 matrix full of 0s except for the central column, which is full of 1s); neurons using these weights will ignore everything in their receptive field except for the central vertical line (since all inputs will get multiplied by 0, except for the ones located in the central vertical line).

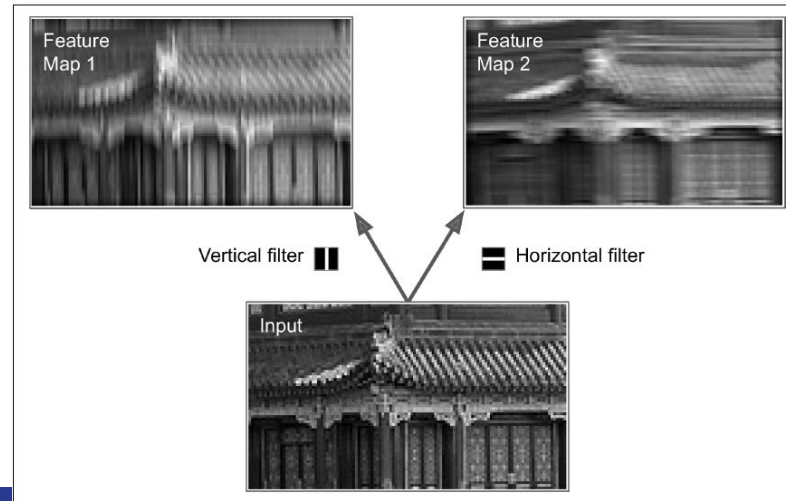


Figure 14-5. Applying two different filters to get two feature maps

Filters (3)

The second: black square with a horizontal white line in the middle. Neurons using these weights will ignore everything in their receptive field except for the central horizontal line.

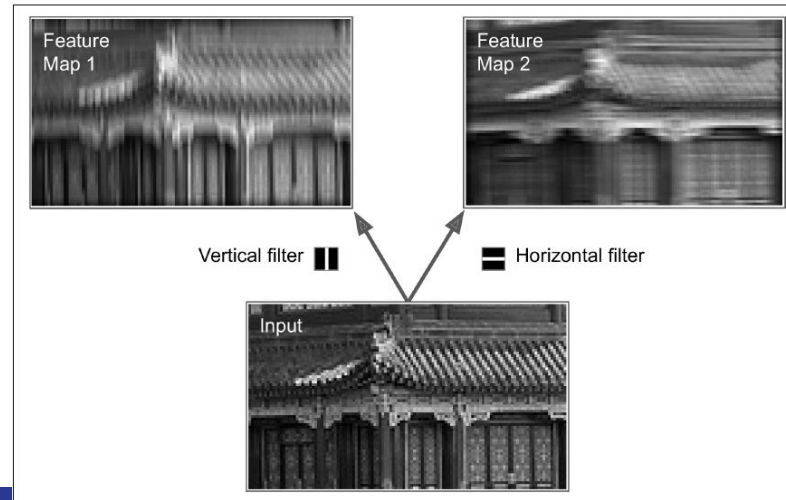


Figure 14-5. Applying two different filters to get two feature maps

Filters (4)

Upper-left *feature map*: vertical white lines get enhanced while the rest gets blurred. Upper-right *feature map*: horizontal white lines get enhanced while the rest is blurred out.

(During training the convolutional layer will automatically learn the most useful filters for its task.)

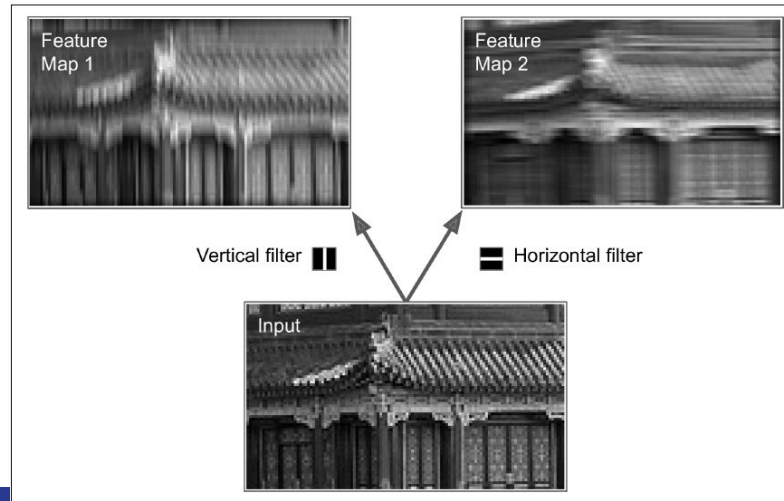


Figure 14-5. Applying two different filters to get two feature maps

Stacking Multiple Feature Maps (1)

2D $\rightarrow$ 3D: a convolutional layer has multiple filters (you decide how many), and it outputs one feature map per filter, so it is more accurately represented in 3D.

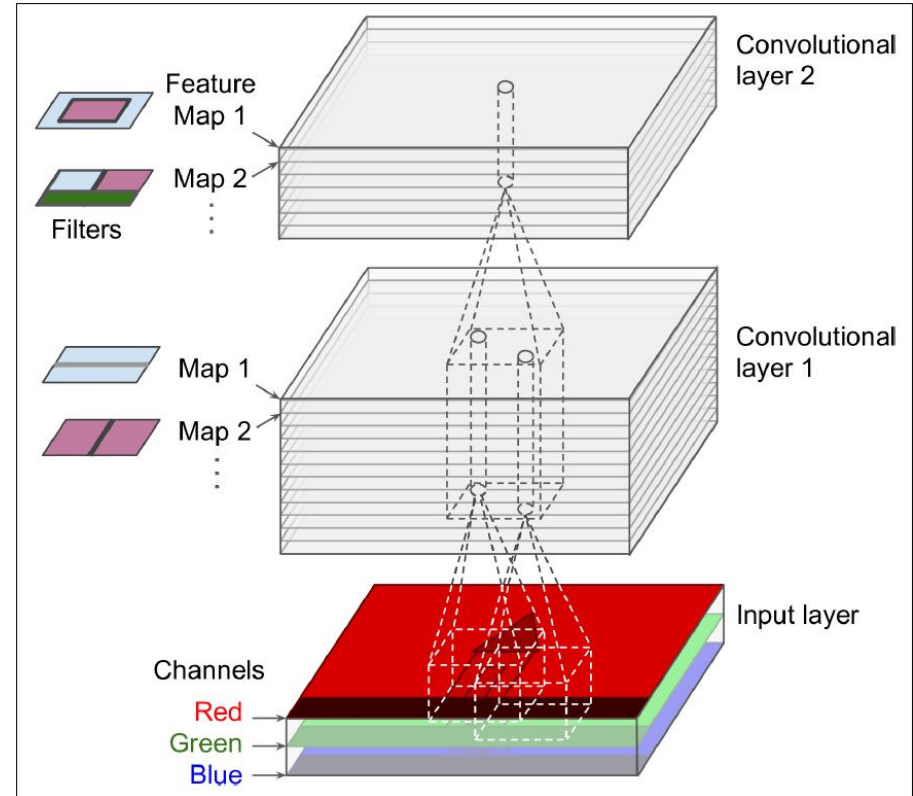


Figure 14-6. Convolution layers with multiple feature maps, and images with three color channels

Stacking Multiple Feature Maps (2)

A convolutional layer simultaneously applies multiple trainable filters to its inputs, making it capable of detecting multiple features anywhere in its inputs.

Input images are also composed of multiple sublayers: one per color channel. Typically 3: red, green, and blue (RGB). Grayscale - 1. Satellite images: some extra, e.g. infrared.

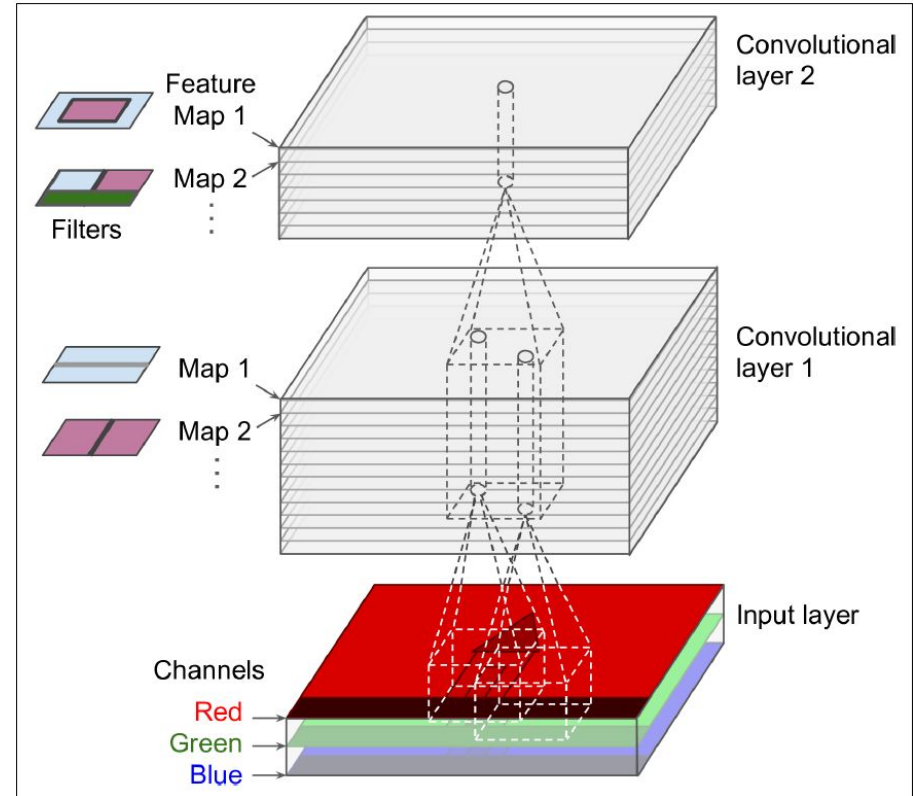


Figure 14-6. Convolution layers with multiple feature maps, and images with three color channels

Stacking Multiple Feature Maps (3)

Neuron in row i , column j of the feature map k in a given convolutional layer l is connected to the outputs of the neurons in the previous layer $l - 1$, located in rows $i \times s_h$ to $i \times s_h + f_h - 1$ and columns $j \times s_w$ to $j \times s_w + f_w - 1$, across all feature maps (in layer $l - 1$). Note that all neurons located in the same row i and column j but in different feature maps are **connected to the outputs of the exact same neurons in the previous layer**.

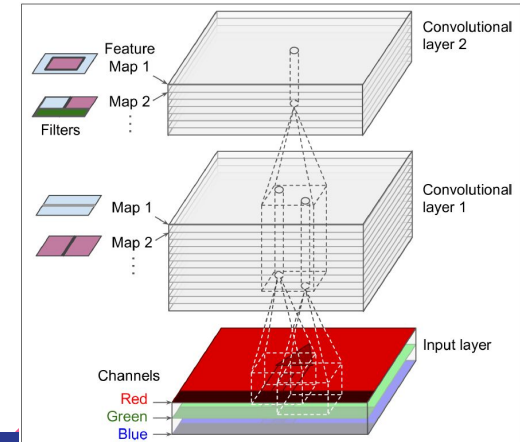


Figure 14-6. Convolution layers with multiple feature maps, and images with three color channels

Stacking Multiple Feature Maps (4)

Equation 14-1. Computing the output of a neuron in a convolutional layer

$$z_{i,j,k} = b_k + \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} \sum_{k'=0}^{f_n-1} x_{i',j',k'} \cdot w_{u,v,k',k} \quad \text{with} \quad \begin{cases} i' = i \times s_h + u \\ j' = j \times s_w + v \end{cases}$$

$z_{i,j,k}$ is the output of the neuron located in row i , column j in feature map k of the convolutional layer (layer l).

s_h and s_w are the vertical/horizontal strides, f_h and f_w are the height/width of the receptive field, and f_n is the number of feature maps in the previous layer (layer $l - 1$).

$x_{i',j',k'}$ is the output of the neuron located in layer $l - 1$, row i' , column j' , feature map k' (or channel k' if the previous layer is the input layer).

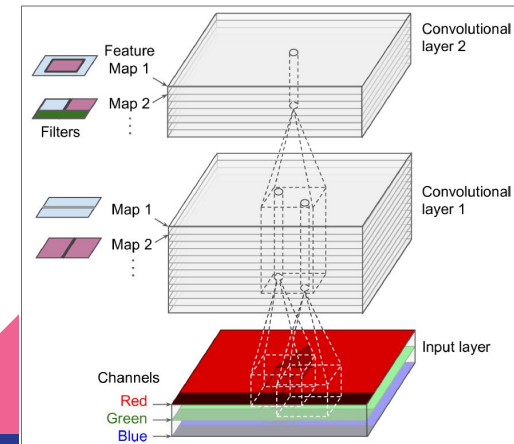


Figure 14-6. Convolution layers with multiple feature maps, and images with three color channels

Stacking Multiple Feature Maps (5)

Equation 14-1. Computing the output of a neuron in a convolutional layer

$$z_{i,j,k} = b_k + \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} \sum_{k'=0}^{f_{n'}-1} x_{i',j',k'} \cdot w_{u,v,k',k} \quad \text{with} \quad \begin{cases} i' = i \times s_h + u \\ j' = j \times s_w + v \end{cases}$$

b_k is the bias term for feature map k (in layer l). You can think of it as a knob that tweaks the overall brightness of the feature map k .

$w_{u,v,k',k}$ is the connection weight between any neuron in feature map k of the layer l and its input located at row u , column v (relative to the neuron's receptive field), and feature map k' .

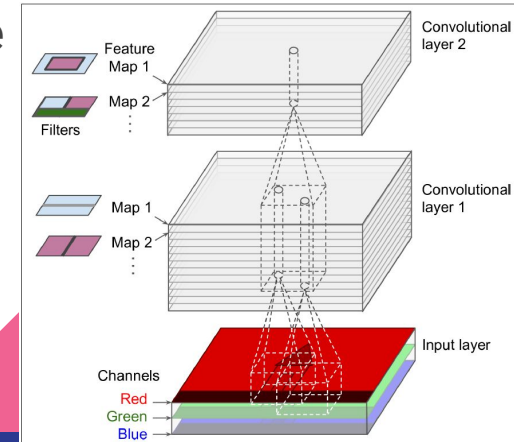


Figure 14-6. Convolution layers with multiple feature maps, and images with three color channels

TensorFlow Implementation (1)

- In TensorFlow, each input image is typically represented as a 3D tensor of shape [height, width, channels].
- A mini-batch is represented as a 4D tensor of shape [mini-batch size, height, width, channels].
- The weights of a convolutional layer are represented as a 4D tensor of shape $[f_h, f_w, f_n, f_n]$.
- The bias terms of a convolutional layer are simply represented as a 1D tensor of shape $[f_n]$.

TensorFlow Implementation (Steps 8-17)

```
from sklearn.datasets import load_sample_image

# Load sample images
china = load_sample_image("china.jpg") / 255
flower = load_sample_image("flower.jpg") / 255
images = np.array([china, flower])
batch_size, height, width, channels = images.shape

# Create 2 filters
filters = np.zeros(shape=(7, 7, channels, 2), dtype=np.float32)
filters[:, 3, :, 0] = 1 # vertical line
filters[3, :, :, 1] = 1 # horizontal line

outputs = tf.nn.conv2d(images, filters, strides=1, padding="SAME")

plt.imshow(outputs[0, :, :, 1], cmap="gray") # plot 1st image's 2nd feature map
plt.show()
```



TensorFlow Implementation (3)

`tf.nn.conv2d()`

- `images` is the input mini-batch (a 4D tensor).
- `filters` is the set of filters to apply (also a 4D tensor).
- `strides` is equal to 1, but it could also be a 1D array with 4 elements, where the two central elements are the vertical and horizontal strides (s_h and s_w). The first and last elements must currently be equal to 1. They may one day be used to specify a batch stride (to skip some instances) and a channel stride (to skip some of the previous layer's feature maps or channels).

TensorFlow Implementation (4)

`tf.nn.conv2d()`

- padding must be either "VALID" or "SAME":
 - "VALID": the convolutional layer does not use zero padding, and may ignore some rows and columns at the bottom and right of the input image, depending on the stride (see Figure 14-7)
 - "SAME": the convolutional layer uses zero padding if necessary. In this case, the number of output neurons is equal to the number of input neurons divided by the stride, rounded up (in this example, $13 / 5 = 2.6$, rounded up to 3). Then zeros are added as evenly as possible around the inputs.

TensorFlow Implementation (5)

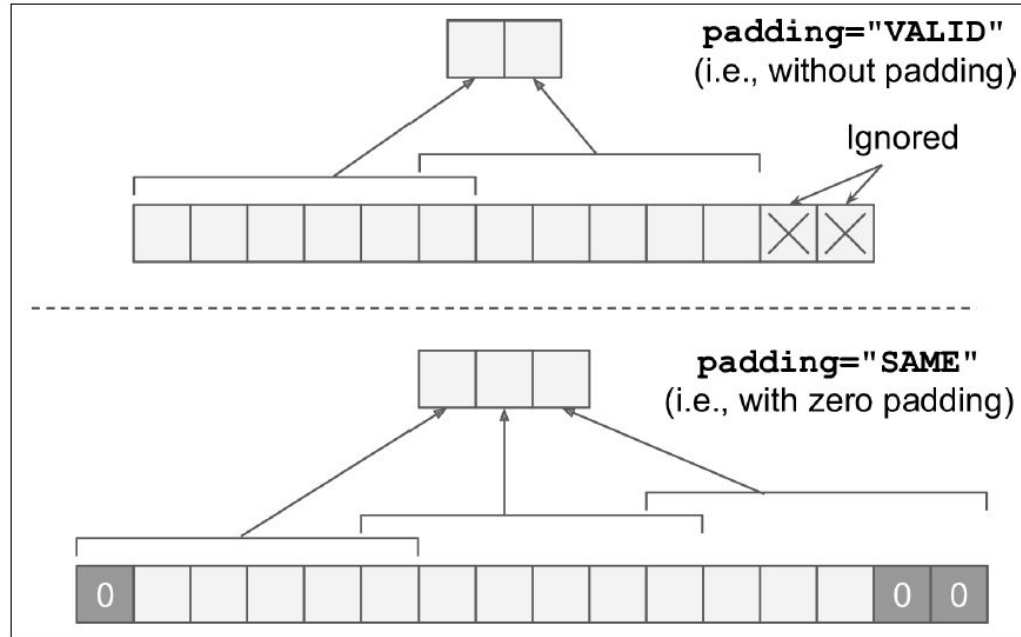


Figure 14-7. Padding options—input width: 13, filter width: 6, stride: 5

TensorFlow Implementation (6)

```
conv = keras.layers.Conv2D(filters=32, kernel_size=3, strides=1,  
                             padding="SAME", activation="relu")
```

Conv2D layer with **32 filters**, each **3 × 3**, using a **stride of 1** (both horizontally and vertically), **SAME** padding, and applying the **ReLU** activation function to its outputs.

Memory Requirements (1)

Convolutional layers require a huge amount of RAM. Especially during training, because the reverse pass of backpropagation requires all the intermediate values computed during the forward pass.

If training crashes because of an out-of-memory error, you can try reducing the mini-batch size. Alternatively, you can try reducing dimensionality using a stride, or removing a few layers. Or you can try using 16-bit floats instead of 32-bit floats. Or you could distribute the CNN across multiple devices.

Memory Requirements (2)

Example: a convolutional layer with 5×5 filters, outputting 200 feature maps of size 150×100 , with stride 1 and SAME padding.

If the input is a 150×100 RGB image (three channels), then the number of parameters is $(5 \times 5 \times 3 + 1) \times 200 = \mathbf{15,200}$ (the +1 corresponds to the bias terms), which is fairly small.

However, each of the 200 feature maps contains 150×100 neurons, and each of these neurons needs to compute a weighted sum of its $5 \times 5 \times 3 = 75$ inputs: that's a total of **225 million float multiplications**.

Moreover, if the feature maps are represented using 32-bit floats, then the convolutional layer's output will occupy $200 \times 150 \times 100 \times 32 = 96$ million bits (12 MB) of RAM. That is just for one instance! If a training batch contains 100 instances, then this layer will use up **1.2 GB of RAM**.

Takeaways

- Principles and definitions for CNNs.